SESSION 9

Capstone Presentations & The Future

9

Show what you built, how you measured it — and where the field goes next.

Jeudi 1er octobre · 11h30–13h15 · 1h45

Prompt & Context Engineering — Stefan Duprey

TODAY

Session 9 — plan & goals

You will be able to

- Deliver a project through Git (PRs, tags)
- Present build, evaluation & reflection
- Give useful peer feedback
- Know where prompt/context engineering goes next

RUN OF SHOW

0:00	Team presentations (delivered via Git)
1:20	Consolidation: cheat sheet
1:35	Where next — auto-optimization, CI/CD
1:40	Closing & resources

Deliver your project through Git

- Merge every feature branch via reviewed pull requests
- Tag a release (v1.0) as your submission
- A clean history tells your project's story
- This is exactly how real teams ship software

WHAT'S NEXT

CI/CD runs your tests automatically on every push — the natural sequel to this course.

FORMAT

Presentation format

Here's how to spend your ten minutes so the work speaks for itself.

1

Build (3 min)

What the app does and a short live demo. Lead with the working thing.

2

Prompt design (2 min)

The key prompts and the iterations that actually moved the numbers.

3

Evaluation (2 min)

Your eval set, the scores, and the judge you used. Show the before/after.

4

Reflection + Q&A (3 min)

Failure modes, one safety concern you found, and questions from the room.

ASSESSMENT

How it's graded

No surprises — here's exactly what we're looking for.

Prompt design

Clear, structured prompts with visible evidence of measured iteration — not a single lucky prompt.

Evaluation

A real eval set, an LLM-as-judge check, and reported before/after numbers. The core of the grade.

Safety

At least one demonstrated risk and a working mitigation. Shows production maturity.

Delivery

Clean Git history, reviewed PRs, a tagged release, and a clear, honest reflection.

PEER REVIEW

Giving good feedback

A quick word on how to be a useful audience for each other.

- **Start with what worked and why**

Specific praise ('the refusal handling was solid') is more useful than generic applause.

- **Ask about one concrete choice**

Probe a single prompt or eval decision rather than vaguely 'have you considered…'.

- **Suggest one improvement, not ten**

A focused, actionable suggestion beats a scattershot list the team can't act on.

AIM

Leave each team with one thing they're genuinely excited to try next — not a scorecard of everything they missed.

CONSOLIDATION

The cheat sheet

If you forget every technique but keep these four habits, you'll still be effective — screenshot this one.

1 Be specific

Role, task, audience, constraints, format. Leave the model no room to guess wrong.

2 Curate context

What's in the window beats exact wording. Retrieve, ground, and cut the rest.

3 Constrain output

Demand a schema; validate and repair. Make output your software can trust.

4 Measure everything

An eval set, one change at a time, with cost and latency beside quality.

OUTLOOK

Where the field is going

Finally, a look at where this is all heading, so what you learned keeps paying off after today.

- **Automatic prompt optimization**

Tools like DSPy tune prompts from data instead of by hand — you specify the goal and a metric, the framework searches.

- **Context engineering over wording tricks**

The centre of gravity keeps moving from clever phrasing to what you put in the window and how you assemble it.

- **More autonomous agents, tighter guardrails**

Capabilities and risks grow together; evaluation and safety become the durable, hard, valuable work.

PARTING THOUGHT

Models will keep changing. A clear spec, curated context, and an eval set will keep working long after today's tricks expire.

THAT'S A WRAP

Thank you — go build.

- Course repo & syllabus: your prompt-engineering-course fork
- Vendor guides: OpenAI & Anthropic prompt-engineering docs
- Deeper systems: the LangChain/LangGraph companion course
- Next step: wire your eval set into CI so tests run on every push